

CGI Capacity Analyzer for Director Workload and RFP Assignment

Freya Kan, Qian Yang, Junxian Lin, Yixiao Jing, Jai
Master of Data Science in Computational Linguistics
University of British Columbia

Abstract

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. We develop a Capacity Analyzer that combines historical CRM opportunity data with proposal and RFP text to support director capacity estimation and RFP assignment. The system is designed as a transparent decision-support tool rather than a replacement for business judgment. In Week 1, we focused on building a reproducible opportunity dataset, defining module boundaries, and preparing the foundations for capacity scoring, dashboarding, and RFP retrieval.

1 Introduction

CGI Atlantic / Media Atlantic currently needs a more consistent and data-informed approach to understanding director workload and assigning new RFPs. Director capacity may be tracked manually or inconsistently, making it difficult to determine who is available, who is close to full capacity, and who is a good fit for a new opportunity.

This project aims to build a CGI Capacity Analyzer that supports two related business decisions. First, the Director Capacity Dashboard will summarize each director's current workload, historical baseline, relative load, capacity score, and capacity label. Second, the RFP Assignment Tool will help estimate RFP effort, retrieve similar historical proposal examples, and recommend suitable directors based on experience, available capacity, service-solution fit, and risk.

The system is not intended to replace stakeholder judgment. Instead, it provides a transparent and reproducible data-based starting point that CGI stakeholders can review, validate, and adjust. This design choice is important because the project does not currently have manually labeled ground truth for director capacity, RFP effort, or correct director assignment.

2 Data

2.1 Data Sources

The project uses two anonymized Excel CRM opportunity exports and one JSON file containing historical proposal and response text. The two opportunity exports are referred to as opps1 and opps2. The proposal data contains historical proposal and response document pairs.

All CGI-provided files are treated as sensitive local data. They are not committed to GitHub. Local credentials and API configuration are also excluded from version control.

2.2 Opportunity Data

Initial EDA indicates that opps1 and opps2 are different views of the same CRM opportunity data. We use opps2 as the base opportunity table because it contains the richer CRM schema and has one unique `opportunity_id` per row in the current export. We use opps1 only to supplement selected fields, including service solution, opportunity product, service-solution revenue, IP, and delivery territory or center.

This distinction is important because the downstream capacity model is expected to operate at the opportunity level. During the Week 1 merge implementation, we found that opps1 contains duplicate `opportunity_id` values. These duplicates likely reflect service-level or product-level detail rather than distinct opportunities. A direct merge from opps1 into opps2 would therefore multiply some opportunity rows and risk double-counting workload in later capacity scoring.

To preserve the intended data grain, the merge pipeline collapses opps1 supplemental fields to one row per `opportunity_id` before joining them to opps2. Duplicate and detail rows from opps1 are preserved in local audit outputs rather than being used to expand the final `opportunity_df`.

Data source	Approx. size	Main contents	Role in project
opps1 CRM export	8,435 rows, columns	20 Opportunity records with supplemental product and service-solution fields	Used to supplement selected fields such as service solution, opportunity product, service-solution revenue, IP, and delivery territory or center.
opps2 CRM export	8,695 rows, columns	33 Richer CRM opportunity schema with owner, status, revenue, date, duration, and proposal-related fields	Used as the base opportunity-level table for opportunity_df.
Proposal / response JSON	19 document pairs	Historical proposal and response text	Used for future RFP chunking, embedding, retrieval, and assignment support.

Table 1: Overview of the main project data sources used in Week 1. The files are stored locally and are not committed to GitHub because they contain CGI-provided data.

2.3 Proposal and RFP Text Data

The proposal JSON contains historical proposal and response document pairs. These documents are text-heavy and many are too long to embed as single inputs. Therefore, the planned RFP pipeline will chunk proposal text before creating embeddings and storing them in a local vector store.

The proposal text will support the RFP Assignment Tool by enabling retrieval of similar historical examples. For a new RFP, the system will retrieve related proposal chunks and combine this semantic evidence with structured opportunity features such as director experience, available capacity, service-solution fit, and risk.

2.4 EDA Findings and Design Implications

The EDA directly influenced several early design decisions. First, because opps2 has the richer CRM schema and preserves the opportunity-level grain more cleanly, it is used as the base table for opportunity_df. Second, because opps1 contains useful service and product fields but also duplicated opportunity IDs, it is treated as a supplemental source rather than a second base table. Third, because the proposal documents are long, the RFP pipeline will use chunking before embedding. Finally, because the data does not contain labeled capacity categories or gold-standard RFP assignments, the initial system will use transparent formula-based scoring and retrieval-based recommendation instead of supervised learning.

3 System Overview

The project has two core deliverables. The first is a Director Capacity Dashboard. This dashboard will use historical CRM opportunity data to estimate each director’s current workload, compare

the current workload against the director’s historical baseline, and display capacity labels such as *Available*, *At Capacity*, and *Overextended*.

The dashboard will also expose drivers of the score, including open opportunities, late-stage opportunities, weighted pipeline, and inferred delivery commitments. This is important because stakeholders need not only a final capacity label, but also an explanation of why a director appears available or overextended.

The second deliverable is an RFP Assignment Tool. For a new RFP, the tool will estimate effort, retrieve similar historical proposal examples, and recommend directors based on experience, available capacity, service-solution fit, and risk. The retrieval component is planned to use chunked proposal text, embeddings, and a local vector store.

The current architecture is modular. The opportunity cleaning pipeline produces a local opportunity_df. The capacity scoring module will consume this dataframe to compute director-level workload and capacity metrics. The Streamlit dashboard will display capacity metrics and provide an interface for RFP assignment. The RFP pipeline will preprocess proposal text, create chunks, and support retrieval over historical examples.

4 Week 1 Implementation Progress

During Week 1, the team focused on building the first reproducible data foundation and creating skeletons for the main system components. The goal was not to finalize the full capacity model, dashboard, or RFP assignment tool. Instead, we prioritized stable data structures, module boundaries, and interfaces that can support later development.

The work was divided across five roles. Yixiao focused on opportunity data merging, schema

Merge statistic	Value
opps1 rows	8,435
opps2 rows	8,695
opps1 unique IDs	7,679
opps2 unique IDs	8,695
Matched opportunity IDs	7,628
opps1-exclusive IDs	51
opps2-only IDs	1,067
Final opportunity_df rows	8,746
Duplicate IDs in final output	0

Table 2: Week 1 opportunity merge audit summary. The final output preserves opportunity-level grain by avoiding one-to-many expansion from duplicated opps1 records.

comparison, and duplicate / unmatched record summaries. Kian focused on validation of revenue, date, duration, probability, status, and sales-stage fields. Lyken focused on the capacity scoring skeleton and director table prototype. Freya focused on the Streamlit dashboard skeleton and RFP page placeholder. Jai focused on the RFP preprocessing and vector store skeleton.

For the opportunity data merge, we implemented a reproducible pipeline that loads the two anonymized CRM Excel exports, normalizes column names, compares schemas, uses opps2 as the base opportunity-level table, and supplements selected fields from opps1. The pipeline uses `opportunity_id` as the required join key.

A key Week 1 finding was that opps1 contains duplicate `opportunity_id` rows. These duplicates likely reflect service-level or product-level detail rather than separate opportunities. A direct merge from opps1 into opps2 created one-to-many expansion, which would risk double-counting workload in downstream capacity scoring.

To preserve the intended opportunity-level grain, we collapsed opps1 supplemental fields to one row per `opportunity_id` before merging. Duplicate and detail rows from opps1 are preserved in local audit outputs instead of multiplying the final `opportunity_df`. The final local `opportunity_df` contains 8,746 rows, matching the expected no-expansion count of 8,695 opps2 base rows plus 51 opps1-exclusive opportunity IDs. The final merged dataset contains zero duplicate `opportunity_id` rows.

All generated processed data files are kept local and are not committed to GitHub because they are derived from sensitive CGI data. The reproducible code and tests are committed instead, so other team members with local access to the source files can

regenerate the same outputs.

5 Current Methodology Plan

The capacity model will be transparent and formula-based. Current workload will combine sales pipeline load and inferred delivery load. Sales pipeline load will use CRM fields such as sales stage, probability, estimated revenue, and project duration. Delivery load will use won opportunities and date or duration fields as a proxy for active delivery work.

Each director's current load will be compared against their own historical baseline. This is important because different directors may normally carry different amounts of work. The dashboard will keep both `capacity_score_owner` and `relative_load_owner`. The capacity score will provide a bounded headroom metric for the main dashboard display, while relative load will help show how far a director is above or below their own historical baseline.

For RFP assignment, the planned approach combines retrieval over historical proposal text with structured opportunity features. Historical proposals will be chunked, embedded, and stored in a local vector store. For a new RFP, the tool will retrieve similar historical RFPs and combine semantic relevance with structured experience signals. Experience is expected to be weighted most heavily, followed by capacity and service-solution fit.

6 Project Management and Next Steps

The Week 1 plan prioritized the data foundation because later scoring, dashboarding, and RFP assignment depend on reliable cleaned opportunity data. In parallel, the team also began module skeletons so that Week 2 work can connect to stable interfaces instead of starting from scratch.

In Week 2, the main focus will be historical baseline construction, current capacity scoring, configurable labels and thresholds, and initial dashboard integration. The cleaned `opportunity_df` will support validation and scoring work. The team will also continue refining assumptions around revenue fields, date fields, probability, status, sales stage, and owner-level aggregation.

This report will be updated weekly. Each week, the team will add implementation progress, design decisions, validation findings, risks, and next steps. This ensures that the final report can be assembled

from time-stamped development notes rather than written only at the end of the project.

7 Current Assumptions and Risks

Several assumptions and risks remain at the end of Week 1. First, `opportunity_id` is treated as the required join key for opportunity-level merging. Second, `opps2` is treated as the base CRM opportunity table, while `opps1` is used only for selected supplemental fields. Third, `opportunity_owner` is currently treated as the main director-level entity for MVP capacity scoring, while `opportunity_manager` is retained as optional metadata.

Additional validation is still needed for revenue, date, duration, probability, status, and sales-stage fields before they are used in capacity scoring. Since the project does not have ground-truth labels for director capacity or RFP assignment correctness, evaluation will rely on stakeholder review, threshold sensitivity checks, and case studies rather than standard supervised-learning metrics.

8 Conclusion

This Week 1 draft establishes the project motivation, dataset description, architecture plan, and initial implementation progress. The main completed data foundation task is a reproducible opportunity merge pipeline that preserves opportunity-level grain and generates a local `opportunity_df`. Future work will build on this foundation to implement capacity scoring, dashboard visualization, RFP retrieval, and recommendation ranking.